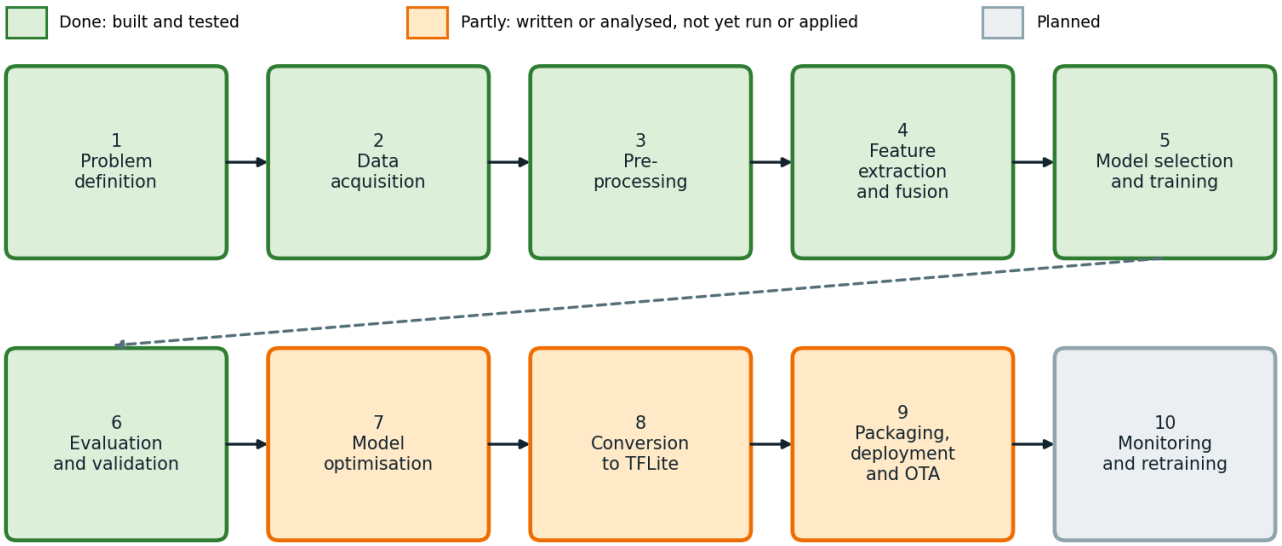
Task D3 — The 10-Stage Pipeline Mapping

Final Report Section 3 | LogiEdge | FreightBridge Logistics Pvt. Ltd. | Component D, Module 4

At a glance

Item	Summary
Mapping	LogiEdge mapped to all 10 stages of the Edge ML pipeline, each described for this system in one or two sentences
Status	Stages 1–6 are done and tested; stages 7–9 are partly done (analysed or written, not yet run or applied); stage 10 is planned
Stage names	The 10 stage names below are the standard edge ML sequence. I do not have the Module 4 list, so please check they match (Section 3)

1 The pipeline at a glance



2 Stage-by-stage mapping

#	Stage	What happens in LogiEdge	Evidence	Status
1	Problem definition and requirements	FreightBridge needs on-truck classification of cargo state as Normal, Warning or Critical, with an alert inside 90 s and no dependence on cellular coverage, within a 10 W AI budget for 85 trucks now and 265 later. The cloud was ruled out because seven 35–90 minute coverage gaps would leave it blind and raw telemetry would cost about ₹25.99 per truck per day.	A1, A2, B1	Done
2	Data acquisition	A Python simulator publishes cargo temperature (1 Hz), compressor vibration RMS (0.5 Hz) and door events to a local Mosquitto broker in four modes: none, temp_drift, vibration and combined. All data so far is simulated; no real truck sensor is connected yet.	C1: sensor_simulator.py	Done (simulated data)
3	Data preprocessing	Each stream gets a 5-sample moving average, is cut into 30 s windows every 10 s, and is z-scored with statistics from 10 minutes of clean Normal data saved in training_stats.npy, which is only ever loaded and never recomputed. A window with under 80% of its samples is skipped rather than filled.	C2: preprocessing.py	Done

#	Stage	What happens in LogiEdge	Evidence	Status
4	Feature extraction and fusion	Each sensor is summarised on its own (temperature mean, standard deviation and °C/min rate; vibration RMS, peak and kurtosis), then the two blocks are concatenated into one 6-value vector. Feature-level fusion was chosen over data-level and decision-level because it matches their accuracy at far lower compute and needs no hand-written cross-sensor rule.	C2, C3	Done
5	Model selection and training	A 6→32→16→3 ReLU MLP with 803 parameters was trained on 291 windows from 20 minutes of Normal, 15 of temperature drift and 15 of combined-fault simulator data. It targets the Raspberry Pi 5 with AI HAT+ (7.5 W against the 10 W budget, ₹15,000 per truck).	D1, B1	Done
6	Evaluation and validation	The 88% gate was passed with 98.25% on the required 20% split, and 50 random splits, 5-fold, blocked and independent-run checks stayed at or above 94.7%. They also found that faults starting mid-run are missed 3.9% of the time, which the onset-trained model removes, and Task C2 measured that 3σ-shifted statistics cost 1.3 to 8.4 points.	D1, C2	Done
7	Model optimisation	Task B2's roofline analysis of a 45 MFLOP model showed it is compute-bound on the Pi 5 CPU (2.81 ms) and favours INT8 quantisation and pruning. The trained model has only 803 parameters and already runs far inside the 90 s limit, so no optimisation has been applied, and quantisation must be validated first because normalised inputs reach about 1,300.	B2	Partly (analysed, not applied)
8	Model conversion	convert_to_tflite.py rebuilds the trained weights as a float32 TFLite model and refuses to write it unless Keras and TFLite match the numpy model on every dataset window. It has not been run yet, because TensorFlow was not available to me.	D2: convert_to_tflite.py	Partly (written, not run)
9	Packaging, deployment and OTA	Preprocessing, inference and the MQTT publisher run in a python:3.11-slim container that publishes to logibridge/trucks/{truck_id}/inference, selects its model with MODEL_PATH and copies the model as the last layer. An OTA model update then resends only that layer, estimated at 16 KB instead of 65 MB per truck, a 99.98% saving for 85 trucks; the image itself has not been built yet.	D2: Dockerfile, ota_demo.sh	Partly (written, not run)
10	Monitoring, drift and retraining	Planned: every result already carries class, confidence, the six features and the model hash, so the operations centre can watch for feature drift against training_stats.npy and falling confidence, which matters because Task C2 showed a 3σ shift costs up to 8.4 points. Retraining on real truck data would be shipped through the same model-layer OTA path.	Problem statement (MLOps); D2 payload	Planned

3 Notes and items to confirm

- **Stage list.** I used the standard 10-stage edge ML sequence: problem definition, data acquisition, preprocessing, feature extraction, model selection and training, evaluation, optimisation, conversion, packaging and deployment, then monitoring and retraining. If Module 4 names or orders them differently, send me the list and I will relabel; the content of each row carries over.
- **Status is stated as it is today.** Stages 7–9 become “Done” once you run convert_to_tflite.py, build the image with ota_demo.sh and (if wanted) apply INT8 quantisation; stage 10 needs the monitoring component. I will update the table with your results.
- **Simulated data.** Stages 2, 5 and 6 rest on simulator data, so accuracy figures are not yet evidence of real-truck performance.
- **Figures quoted** (98.25%, 3.9%, 99.98%) come from Tasks D1 and D2; the 16 KB and 65 MB sizes are my estimates until measured.